Benchmarking of zk-STARK and zk-SNARK in Privacy-Preserving Federated Learning Scenario

Lars Blütecher Holter[1][0009-0004-0932-4447]

Norwegian University of Science and Technology - NTNU, Norway
larsern99@hotmail.com

Abstract. Sharing sensitive information for the purpose of building machine learning models is often prohibited due to technical, legal, or privacy aspects. Federated learning is a technology where distributed participants collaborate to build machine learning models while at the same time protecting sensitive private information. The challenge is to enable distributed computing without sharing data. One of the solutions is in non-interactive zero-knowledge proof algorithms. Zero-knowledge proofs enable verification of computational correctness without revealing private data, and thus zero-knowledge technology addresses critical privacy challenges in federated learning. Although zk-SNARKs have been applied to federated learning systems, no empirical comparison between zk-SNARKs and zk-STARKs has been conducted, and thus we provide an evaluation framework for benchmarking zero-knowledge algorithms in a federated learning scenario. Our experiments demonstrate that zk-STARKs outperform zk-SNARKs for privacy-preserving federated learning applications, achieving significant performance advantages. The implementation of zk-STARK exhibits sublinear $O(nlogn)$ scaling characteristics, requiring only 14.37 seconds for the batch size of 40, while zk-SNARKs demonstrate $O(n^2)$ behavior, with 1011.68 seconds for the same workload. Memory efficiency analysis reveals that zk-STARKs consume 2.9× to 6.4× less memory. Although zk-STARK proofs are larger (1359× to 1671× the size of zk-SNARK proofs), this trade-off becomes increasingly acceptable for computationally intensive applications where the performance gains outweigh storage costs.

Keywords: zero-knowledge · zk-STARK · blockchain · federated learning · zk-SNARK · benchmark

1 Introduction

Federated learning, introduced by Google in 2016, enables collaborative model training without sharing raw data by distributing computations across decentralized devices [7]. However, FL systems remain vulnerable to malicious participants who may submit invalid model updates designed to poison the global model, or honest but curious participants may extract private information. Existing FL frameworks have integrated zk-SNARKs to enforce verifiable training

and aggregation [3, 6], but these implementations face challenges: (i) zk-SNARKs require a trusted setup that creates potential security vulnerabilities, (ii) due to the reliance on elliptic curves they are not quantum safe, and (iii) exhibit poor scaling as model complexity increases, making proof generation prohibitively expensive for complex AI workloads.

In contrast, zk-STARKs employ a transparent setup that eliminates trust assumptions and offer quasi-linear scaling in proof generation time. In addition, the underlying technology relies on collision-resistant hash functions and the FRI protocol, providing post-quantum security guarantees. Theoretical analysis suggests that zk-STARKs are potentially better suited for AI-centric workloads [8]. However, despite these theoretical predictions, no prior work has implemented a complete zk-STARK-based FL system and directly compared its performance to zk-SNARKs. Our work addresses this research gap by providing a comprehensive empirical evaluation of zk-STARK versus zk-SNARK in federated learning.

Our contributions include: (1) proof-of-concept implementation of federated learning using zk-STARKs that provides empirical comparison between two zero-knowledge algorithms, zk-STARKS and zk-SNARKs; (2) custom AIR constraints encoding neural network operations for privacy-preserving federated learning; (3) comprehensive performance analysis demonstrating 70× speedup and superior memory efficiency; (4) validation of theoretical predictions about zk-STARK superiority for computationally intensive AI applications.

2 Background and Related Work

zk-SNARKs provide compact proofs with fast verification. However, they require a trusted setup where cryptographic parameters are generated, e.g. through multi-party computation [2]. If participants retain secret randomness from the trusted setup, so-called toxic waste, they could forge proofs, compromising system security. `Groth16` produces constant-size proofs regardless of computation complexity [5], but reliance on elliptic curves introduces potential vulnerability to quantum attacks.

zk-STARKs eliminate the need for trusted setup, and use collision-resistant hash functions and the FRI protocol. The transparent setup ensures that participants can independently verify the system parameters without trusting external parties. zk-STARKs provide post-quantum security, since they depend on hash functions rather than number-theoretic assumptions vulnerable to quantum attacks.

Haimerl's thesis presented a blockchain-based FL architecture using zk-SNARKs for on-chain verification [6]. IoT edge devices compute local updates and generate zk-SNARK proofs submitted to an Ethereum smart contract. The implementation demonstrated zero-knowledge integration, but faced scalability challenges due to high gas costs and quadratic scaling in circuit compilation.

Ebrahimi et al. developed VeriBlock-FL, that can perform proof generation off-chain while maintaining on-chain verification [3]. This design reduces gas

costs but still faces fundamental scalability limitations due to the underlying zk-SNARK technology.

A systematic literature review by Oude Roelink et al. found 31 zk-SNARK deployments and none zk-STARK implementations in practical systems [8]. Although El-Hajj et al. benchmarked zk-SNARKs and zk-STARKs using MiMC hash computations, their evaluation used only simplified circuits rather than realistic AI workloads [4].

3 System Design and Implementation

3.1 Architecture Overview

Our system implements an off-chain federated learning framework where clients generate zk-STARK proofs for local training, and the aggregator produces verifiable proof for averaging and aggregating local models. The use of zk-STARKs for both, training and aggregation, eliminates blockchain overheads. The setup also enables us to compare zk-SNARKs and zk-STARKs in a realistic federated learning workload.

The system workflow follows these key steps:

1. **Train local model & generate zero-knowledge proof**: Each client performs local training using the neural network implementation and generates a zk-STARK proof using the `TrainingUpdateAir` constraints.
2. **Send local model & and the proof**: Clients transmit their model updates along with the corresponding ZKPs directly to the aggregator. There is no need for any blockchain infrastructure.
3. **Verify ZKPs off-chain**: The aggregator validates all received proofs, ensuring the computational integrity of local models. This allows the aggregator to verify the data before aggregating, and without accessing any private data.
4. **Update global model & generate ZKP**: Using the federated averaging algorithm (`FedAvg`), the aggregator computes a new global model and generates its own proof for the aggregation process.
5. **Distribute updated global model**: The aggregator sends the updated global model along with the proof back to all participating clients for the next training round. Cients can also verify that the aggregation was done correctly.

This design ensures that every computational step is cryptographically verifiable while maintaining complete privacy of local training data, as illustrated in Figure 1.

3.2 zk-STARK Implementation

Our zk-STARK system is implemented in `Rust` and uses the `Winterfell` library. Our implementation relies on two custom Arithmetic Intermediate Representation (AIR) modules encoding federated learning computational constraints.

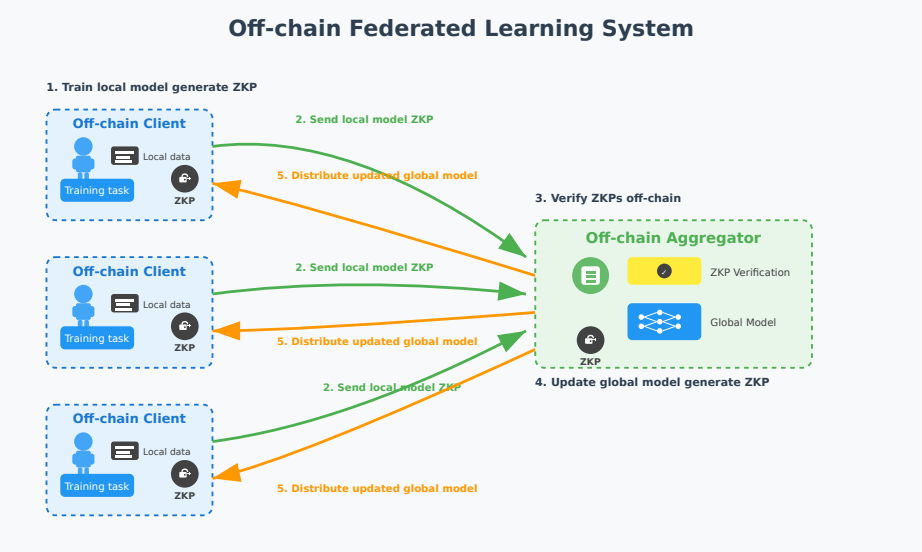

Fig. 1. Overview of the off-chain federated learning system

TrainingUpdateAir encodes local stochastic gradient descent (SGD) operations, including forward propagation, backward propgation, and gradient descent. The core constraint ensures correct parameter updates:

$$\text{weight}^{(t)} - \text{gradient} \cdot \eta - \text{weight}^{(t+1)} = 0$$

GlobalUpdateAir encodes federated averaging, enforcing constraints like:

$$k \times \text{new\_global} - k \times \text{global} - \text{update} = 0$$

where k is the scaling factor for aggregation.

3.3 Finite Field Arithmetic

A critical challenge for using zk-STARKs is the representation of real-valued parameters within finite fields. We have developed a fixed-point encoding using a scaling factor 10^6 and a signed-pair representation $(|v|, \text{sign})$. A cleanse function maps signed pairs to field elements:

$$\text{cleanse}(m, s) = \begin{cases} m & \text{if } s = 0 \\ \text{MAX} - m + 1 & \text{if } s = 1 \end{cases}$$

We implemented generic arithmetic operations handling sign propagation correctly while maintaining computational correctness in finite field operations.

3.4 Zero-Knowledge Properties

We achieve zero-knowledge through one-time pad masking that conceals model parameters while preserving computational relationships. Masking is applied through field addition: masked_param = original_param+random_mask. Since masks are uniformly distributed, this provides information-theoretic privacy guarantees.

4 Experimental Setup

For the experiments we have used the *Daily and Sports Activities* dataset containing IoT sensor data [1]. Following VeriBlock-FL methodology, we reduced features from 45 to 9 dimensions and consolidated 19 activities to 6 categories. We used a single-layer neural network with 9 inputs and 6 outputs.

Our benchmarking framework conducted systematic evaluation across batch sizes between 1 to 40, executing each configuration 11 times to obtain statistical reliability. We measured execution time (total and phase-wise), memory consumption (peak and average), and recorded the proof sizes. For zk-SNARK comparison, we used the VeriBlock-FL codebase with identical parameters.

The framework implements multiple timing sources, memory monitoring through application and system-level tools, automated outlier detection using z-score methodology, and statistical analysis including confidence intervals.

All experiments used MacBook Pro (2023) with Apple M2 Pro processor, 16 GB memory, running macOS Sequoia 15.4.1.

5 Results and Analysis

5.1 Overall Performance

Our evaluation reveals significant performance advantages for zk-STARKs across all batch sizes. At batch size 1, zk-STARKs complete in $4.71 \pm 3.6\%$ (coefficient of variance, CV, reported as percentage) seconds versus $9.91 \pm 0.36\%$ seconds for zk-SNARKs (2.1× speedup).

This gap widens substantially at batch size 40, where zk-STARKs require only $14.37 \pm 4.3\%$ seconds while zk-SNARKs need $1011.68 \pm 0.41\%$ seconds, yielding a 70.4× **speedup**.

zk-STARK execution demonstrates sub-linear scaling, growing only 3.0× across the tested range, consistent with $O(nlogn)$ complexity. zk-SNARKs exhibit polynomial scaling, increasing 102× and approaching $O(n^2)$ scaling behavior.

5.2 Phase-Level Analysis

For zk-SNARKs, compilation emerges as the primary bottleneck, escalating from 4.83 seconds (batch 1) to 717.47 seconds (batch 40). This represents 149× increase representing 70.9% of total execution at scale. Setup grows from 2.49

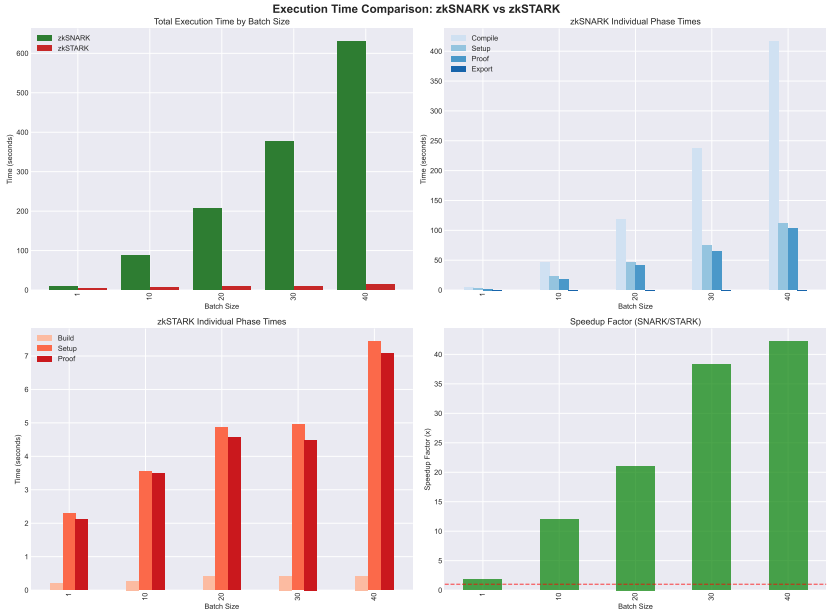

Fig. 2. Execution time comparison between zk-SNARK and zk-STARK systems across different batch sizes, showing total time and phase-wise breakdown.

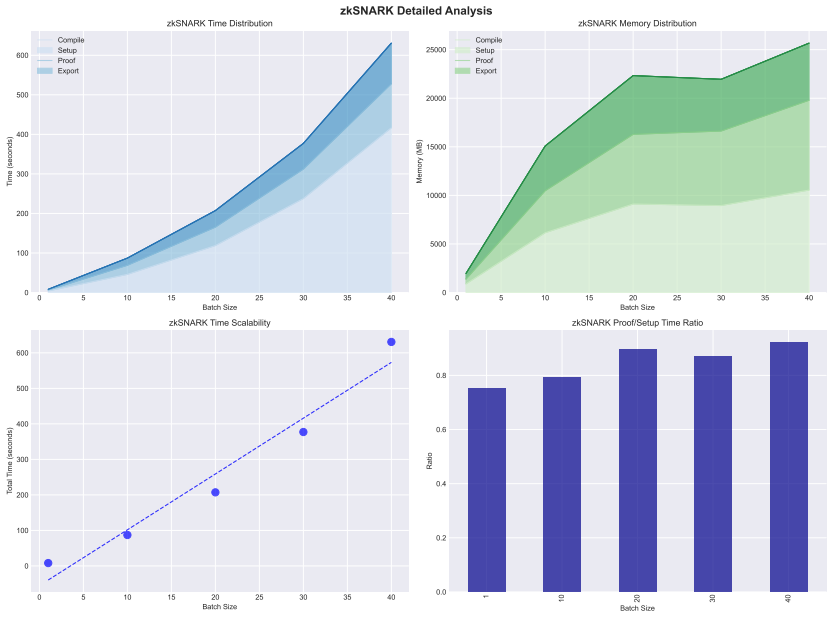

Fig. 3. Detailed analysis of zk-SNARK performance showing time and memory distribution across different phases (compile, setup, proof, export) and scalability characteristics.

to 146.65 seconds (59× increase), while proof generation increases from 1.82 to 122.89 seconds (68× increase).

zk-STARKs maintain balanced performance across phases. Build phase increases minimally from 0.21 to 0.45 seconds (2.1× increase). Setup and proof phases show controlled growth: 3.2× (2.25 to 7.17 seconds) and 3.0× (2.25 to 6.75 seconds) respectively, with no single phase dominating.

5.3 Memory Efficiency

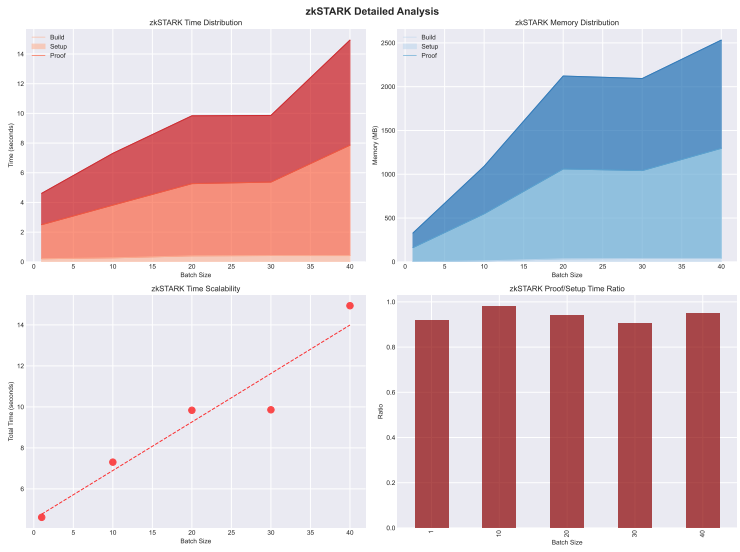

Fig. 4. Detailed analysis of zk-STARK performance showing time and memory distribution across different phases (build, setup, proof) and scalability characteristics.

zk-STARKs consistently require 2.9× to 6.4× less memory than zk-SNARKs. At batch size 40, zk-STARKs peak at 1.42 GB compared to 6.25 GB for zk-SNARKs. Memory scaling favors zk-STARKs with controlled logarithmic growth (setup increases 8.8×), while zk-SNARKs exhibit near-polynomial scaling (setup increases 10.9×) creating severe resource requirements.

5.4 Proof Size Trade-offs

The primary zk-STARK disadvantage is the proof size. zk-STARK proofs range from 1.48 MB to 1.91 MB, representing 1359× to 1671× larger than zk-SNARKs (always under 1 KB). However, proof size growth is sub-linear: despite 40× computational increase, zk-STARK proof size increases only 29.1%.

In federated learning scenarios where verification frequency is low relative to proof generation, computational efficiency gains significantly outweigh storage costs. Clients generate proofs continuously but verification occurs only during periodic aggregation.

5.5 Validation of Intrinsic Performance

To ensure performance differences stem from algorithmic characteristics rather than implementation artifacts, we analyzed zk-SNARKs excluding compilation. Even optimized, zk-STARKs maintain substantial 20.5× advantage (14.4 vs 294.2 seconds at batch size 40), confirming intrinsic superior scaling.

Statistical reliability analysis shows consistent measurements across both systems. zk-STARKs demonstrate moderate temporal variance (CV 2.3% to 5.1%) and stable memory variance, while maintaining highly consistent proof sizes (CV under 0.5%).

6 Discussion and Implications

Performance Implications. The empirical evidence validates theoretical predictions about zk-STARK superiority for AI applications. The performance advantages demonstrate transformative differences that fundamentally alter privacy-preserving computation viability at scale. Memory efficiency characteristics enable deployment on standard infrastructure without specialized hardware, making privacy-preserving AI accessible to resource-constrained organizations.

The transparent setup eliminates operational complexity and security vulnerabilities from trusted setup ceremonies. Combined with post-quantum security, these characteristics position zk-STARKs as superior for applications requiring long-term security assurance.

Scalability and Cost-Benefit Analysis. Sub-linear scaling characteristics suggest zk-STARKs become increasingly advantageous as AI models grow in complexity—precisely modern AI development trajectory. Organizations operating large-scale privacy-preserving AI systems would realize substantial cost savings from 70.4× computational reduction, even accounting for additional storage costs.

The demonstrated efficiency improvements translate to significant energy savings supporting environmentally sustainable deployment of privacy-preserving AI technologies. The implementation using general-purpose programming languages demonstrates practical viability across diverse systems and platforms.

Practical Deployment Considerations. For applications where verification frequency is low relative to proof generation—typical in federated learning—computational efficiency gains significantly outweigh storage costs. The sub-linear proof size growth suggests this trade-off becomes increasingly favorable as system scale increases.

Balanced phase performance indicates robust scalability without algorithmic bottlenecks that would limit applicability to complex workloads. This positions

zk-STARKs as foundational technology for next-generation privacy-preserving AI systems that can scale to meet demands of increasingly complex machine learning applications.

7 Limitations

This study acknowledges several limitations affecting the interpretability and generalizability of its findings.

Internal Validity.

Implementation asymmetries: The zk-SNARK baseline utilized ZoKrates (a high-level toolkit), while the zk-STARK implementation employed Winterfell in Rust. Toolchain differences may introduce performance variations unrelated to protocol efficiency.

Simplified arithmetic: Signed arithmetic in finite fields was implemented via fixed-point encoding rather than bit decomposition, potentially inflating constraint complexity.

Neural network simplicity: The single-layer architecture (9 inputs $\times$ 6 outputs) does not reflect computational demands of deep learning models.

External Validity.

Dataset constraints: Experiments used the *Daily and Sports Activities* dataset (9 features, 6 classes), limiting extrapolation to high-dimensional data (e.g., images, text).

Hardware specificity: Benchmarks executed exclusively on Apple M2 Pro (ARM architecture); results may differ on x86 CPUs or GPUs.

Scale limitations: Batch sizes capped at 40 samples; behavior at web-scale FL (1,000+ clients) remains unverified.

Reliability.

Statistical robustness: Eleven runs per configuration ensured stable central estimates ($CV < 5.1\%$ for temporal metrics), but rare anomalies (e.g., the 5.7% runtime dip for zk-STARKs at batch 30) warrant further investigation.

Environmental consistency: Memory monitoring relied on macOS APIs; cross-platform verification is needed to exclude OS-specific artifacts.

These limitations constrain direct extrapolation to industrial-scale FL but establish a foundational framework for future benchmarking.

8 Conclusion

Our work provides a comprehensive empirical validation of zk-STARK performance as compared to zk-SNARKs in privacy-preserving federated learning. Our systematic evaluation demonstrates transformative performance advantages, 70.4$\times$ speedup and 2.9$\times$ to 6.4$\times$ memory efficiency—while providing transparent setup and post-quantum security.

The sublinear scaling characteristics make zk-STARKs optimal for AI applications where computational demands continue to grow. While proof sizes

remain larger, demonstrated efficiency gains outweigh storage costs for large-scale applications, particularly as the relative penalty grows sub-linearly.

These findings challenge prevailing assumptions about zero-knowledge proof selection and suggest a paradigm shift for computationally intensive applications. The work establishes zk-STARKs as the preferred technology for privacy-preserving machine learning requiring scalability and performance, with implications extending to any domain requiring privacy-preserving verification of complex computations.

References

1. Barshan, B., Altun, K.: Daily and Sports Activities. UCI Machine Learning Repository (2010), DOI: https://doi.org/10.24432/C5C59F
2. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. p. 326–349. ITCS '12, Association for Computing Machinery, New York, NY, USA (2012). https://doi.org/10.1145/2090236.2090263, https://doi.org/10.1145/2090236.2090263
3. Ebrahimi, E., Sober, M., Hoang, A.T., Ileri, C.U., Sanders, W., Schulte, S.: Blockchain-based federated learning utilizing zero-knowledge proofs for verifiable training and aggregation. In: 2024 IEEE International Conference on Blockchain (Blockchain). pp. 54–63 (2024). https://doi.org/10.1109/Blockchain62396.2024.00017
4. El-Hajj, M., Oude Roelink, B.: Evaluating the efficiency of zk-snark, zk-stark, and bulletproof in real-world scenarios: A benchmark study. Information (Switzerland) **15**(8) (2024). https://doi.org/10.3390/info15080463, publisher Copyright: © 2024 by the authors.
5. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) Advances in Cryptology – EUROCRYPT 2016. pp. 305–326. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
6. Haimerl, N.: Blockchain-based federated learning with data verification through zero-knowledge proofs (2022). https://doi.org/10.34726/HSS.2022.90711, https://repositum.tuwien.at/handle/20.500.12708/19921
7. McMahan, H.B., Moore, E., Ramage, D., Hampson, S., y Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data (2023), https://arxiv.org/abs/1602.05629
8. Oude Roelink, B., El-Hajj, M., Sarmah, D.: Systematic review: Comparing zk-snark, zk-stark, and bulletproof protocols for privacy-preserving authentication. Security and Privacy **7**(5), e401 (2024)